

Rosetta

A text-to-SQL system that refuses to answer questions it cannot answer honestly — and measures how often guessing produced a confident wrong number.

Project report, written from first principles. No prior knowledge of text-to-SQL or Indian public finance is assumed.

Commit `efd015b` + uncommitted changes · 14 September 2026 · every figure measured and reproducible from this repository.

How to read this

Part 0 assumes you know nothing at all — not what a database is, not what SQL looks like, not what a language model does. Everything the rest of the report leans on is explained there first, with worked examples. If you already write SQL and have used a language model, skip it.

Parts I and II are background: what the problem is and what the data means. Nothing project-specific until Part III. If you already know what text-to-SQL is, start at Part III.

§15b is a single real question traced through the whole system, with the actual output. If you prefer to see a thing work before reading how it is built, start there.

Part VI is the section worth reading twice. It records the design decisions that turned out to be **wrong**, and what replaced them. In a project whose subject is confidently wrong numbers, the mistakes are the most informative part of the record.

Part 0 — Starting from zero

Nothing in this part is specific to this project. It explains the ideas the rest of the report leans on, assuming you have never written a line of SQL and have never used a language model. If those are familiar, skip to Part I.

0.1 What a database actually is

A **database** is a program whose one job is to store facts and hand them back on request. The facts live in **tables**, which look like spreadsheets:

budget_allocations

fiscal_year	ministry	be_amount_crore	actual_amount_crore
2019-20	Ministry of Railways	70,250	65,837
2019-20	Ministry of Defence	305,296	316,296
2020-21	Ministry of Railways	70,000	57,891

Three words carry all the weight:

- A **row** is one fact. "In 2019-20, Railways was allocated 70,250 crore."
- A **column** is one attribute that every row has. `fiscal_year` is a column.
- The **schema** is the catalogue: which tables exist, which columns each has, and what type each column holds (a number, a date, text).

The schema is the part that matters here. This project's database has **11 tables and 182 columns**, which is small by real-world standards and still far too much for anyone to hold in their head. A real corporate warehouse has hundreds of tables and thousands of columns, and that gap between "what exists" and "what one person can remember" is the entire problem this project is about.

0.2 Reading SQL without knowing SQL

SQL is the language you use to ask a database for something. You do not need to write it to follow this report; you need to read it. It is more regular than it looks, because every query is built from the same handful of clauses:

CLAUSE	PLAIN ENGLISH
SELECT	which columns I want back
FROM	which table to take them from
WHERE	keep only rows matching this condition
GROUP BY	collapse rows into groups, one output row per group
SUM(x) / COUNT(*)	add up a column / count rows
JOIN	glue two tables together on a shared value
ORDER BY	sort the output

So this query:

```
SELECT ministry, SUM(be_amount_crore)
FROM budget_allocations
WHERE fiscal_year = '2019-20'
GROUP BY ministry
```

reads as: *take the budget table, throw away every row that is not 2019-20, put the surviving rows into piles by ministry, and add up the crore column within each pile. One row comes back per ministry.*

That is genuinely most of it. SQL's difficulty is not its grammar.

0.3 The difficulty is knowing what the columns mean

Look again at the table in §0.1. It has `be_amount_crore` and `actual_amount_crore`. Both are money. Both are for the same ministry and the same year. They are different numbers.

Nothing in the schema tells you which one a person means when they say "the budget for Railways in 2019-20". The database will happily add up either. It will never warn you that you picked the one nobody meant.

This is the whole problem in one sentence: a database answers the question you typed, not the question you meant, and it cannot tell the difference.

A person who has worked with this data for a year knows which column to reach for. That knowledge is real, valuable, and written down nowhere. Making it explicit is what a **semantic layer** is (§13), and measuring what it is worth is what this project does.

0.4 NULL: the value that means "we don't know"

Databases need a way to say "there is no value here". That marker is **NULL**, and it is not zero. Zero is a measurement. NULL is the absence of one.

This distinction causes real bugs, because NULL infects every comparison it touches. In ordinary logic a statement is true or false. SQL has **three** outcomes — true, false, and NULL — and a comparison against NULL returns NULL:

```
NULL < 2016      -- not TRUE, not FALSE. NULL.
NOT (NULL < 2016) -- still NULL, which SQL treats as "do not keep this row"
```

The second line is the trap. It looks like it means "keep everything that is 2016 or later, including rows with no date". It does not. It silently discards every NULL row. §36 of this report is a real instance of exactly that bug, in a script written to catch bugs.

0.5 Joins, grain, and how a sum can silently triple

A **join** glues two tables together where they share a value. If one table has a row per ministry per year, and another has a row per ministry per month, joining them matches each annual row against *twelve* monthly rows.

The **grain** of a table is what one row represents. One year? One month? One scheme in one state in one month? Joining tables of different grain duplicates the coarser side:

before the join	after joining to 12 monthly rows
Railways 70,250	Railways 70,250 January
	Railways 70,250 February
	Railways 70,250 March ... and so on

The annual figure now appears twelve times. `SUM(be_amount_crore)` no longer returns 70,250. It returns 843,000. This is called **fan-out**, and it is dangerous precisely because nothing fails — the query runs, returns one tidy number, and that number is wrong by a factor nobody can guess from looking at it. On this project's data, the measured inflation from one such join is **33×**.

(The twelve-row illustration above is a simplified sketch; 33× is the real measured figure from this repository's data.)

0.6 What a language model is

A **large language model** (LLM) is a program that predicts text. Shown some text, it produces what most plausibly comes next, one fragment at a time. It learned to do this from an enormous amount of writing, which is why it can produce fluent SQL: it has seen a great deal of SQL.

Two consequences matter here, and they are the reason this project exists.

It is not looking anything up. When it writes `SELECT SUM(expenditure_crore)`, it has not consulted your database. It has produced the column name that *seems most likely* given everything it was shown. If your real column is `be_amount_crore`, the model has no way to know it guessed wrong.

Plausible and true are indistinguishable in the output. A correct answer and a fabricated one arrive in the same confident tone, with no hedging and no error. This is usually called **hallucination**. The engineering response is not to hope for a better model — it is to build a system where a fabricated answer cannot quietly become the final answer.

0.7 Prompts, temperature, and why "the same question" can give different answers

The text you hand the model is the **prompt**. In this project the prompt is assembled from three things: a description of the database schema, the semantic layer (when it is switched on), and the user's question.

Temperature controls how adventurous the model's choice of next fragment is. At temperature 0 it always takes the single most likely option, which is as close to repeatable as a model gets — and it is what every measurement in this report uses.

"As close as it gets" is not "exactly". The same model at temperature 0 can still produce different output across server restarts, because floating-point arithmetic on a GPU is not guaranteed to combine in the same order every time. That is not a footnote: it is precisely why Part VII measures the **spread** across three runs rather than quoting a single number (§41).

0.8 Running the model on your own machine

There are two ways to use a language model: send your text to somebody's server over the internet (an **API**), or run the model on your own hardware (**local inference**). This project runs locally, through a tool called **Ollama**, using a model called **qwen2.5-coder:7b**.

Decoding that name: it is a model specialised for writing code, with **7 billion parameters** — the internal numbers it learned during training. Bigger models are generally better and need more memory. 7B is small enough to run on an ordinary computer.

The trade is deliberate. A local model writes worse SQL than a large hosted one, but it has no per-request cost, no rate limit, and no key to manage — which matters when the evaluation makes several hundred model calls and is run many times over. And since the project measures the *difference* between two setups rather than absolute quality, a weaker model does not invalidate the finding (§25).

0.9 Benchmarks, and what a single accuracy number hides

A **benchmark** is a fixed set of questions with known-correct answers, used to compare systems fairly. **Spider** and **BEAVER**, named in §3 and §11, are the well-known benchmarks for text-to-SQL.

The usual score is **execution accuracy**: of all the questions, what share produced a query returning the right rows? It is one number, easy to compare, and it hides the thing that matters most. Consider two failures:

the query crashed	the user sees an error and asks again
the query returned 8,412	the user puts 8,412 into a board report

Execution accuracy counts these identically — both are simply "not correct". But a crash is **visible** and costs a retry, while a clean wrong number is **invisible** and propagates. Nobody re-checks a figure that arrived without an error attached.

Splitting those apart is why this report uses six outcome labels instead of one score (§24), and why its headline is the **confidently-wrong rate** rather than accuracy.

0.10 Deterministic, probabilistic, and why it matters here

Something is **deterministic** when the same input always produces the same output. A hand-written rule is deterministic. A language model is **probabilistic** — it samples from likelihoods, so it may answer differently on Tuesday.

This distinction drives the central design decision of the project. The part that writes SQL is probabilistic, because writing SQL from English genuinely needs that flexibility. The part that decides *whether a question can be honestly answered at all* is deterministic — hand-written rules, no model (§21). The reasoning is short: the decision not to answer should not be delegated to the thing you are guarding against.

The database itself is also built deterministically, from a fixed starting number called a **seed**, so that anyone who clones the repository and runs one command gets byte-identical data.

0.11 YAML, in thirty seconds

Three files in this project hold the domain knowledge, written in **YAML** — a plain-text format for structured data, designed to be edited by humans rather than generated by programs. Indentation indicates nesting:

```
be_amount_crore:
  unit: crore
  means: Budget Estimate, as presented in February
  invites: being read as "what was actually spent"
```

That is a name, and three labelled facts about it. Nothing more. It is used here rather than code because these files are meant to be argued over and reviewed by people who are not programmers.

0.12 How to read the experiment

Part VII reports a measurement, and it uses four words in a specific way.

WORD	MEANING
Arm	One configuration being compared. Here there are two: with the semantic layer, and without it.
Baseline	The arm you are trying to beat — here, no semantic layer.
Ablation	An experiment that changes exactly one thing and holds everything else identical, so any difference can only be caused by that one thing.
Spread	The gap between the highest and lowest result across repeated runs. It tells you how much of a difference is real and how much is noise.

The last one is the one people skip. If a change improves a score by 3 points but the score wanders by 5 points on its own between runs, the change has not been shown to do anything. A result quoted without its spread is not yet a claim. That is why every headline in Part VII is a mean of three runs with its range printed next to it.

Part I — Foundations

1. What a database query is

Information in a database sits in tables — rows and columns, like a spreadsheet with rules. To get anything out you write **SQL**:

```
SELECT SUM(be_amount_crore)
FROM budget_allocations
WHERE fiscal_year = '2019-20'
```

Read it as: *add up the `be_amount_crore` column, across every row where the year is 2019-20.*

SQL takes weeks to learn. Learning a *particular organisation's* database takes months — not because the syntax is hard, but because you have to learn what the columns mean, which ones are safe to add together, and which of the six columns with "amount" in the name is the one people actually quote.

2. What text-to-SQL is

Text-to-SQL removes that barrier: you ask in English, a language model writes the SQL, the database answers.

It is one of the most commercially attractive uses of language models, for a simple reason — every organisation has a database, and almost nobody in the organisation can query it. The people who want the numbers and the people who can extract them are different people, and the gap between them costs real time.

3. Why it works in demos and fails in production

Text-to-SQL demos are reliably impressive. Production deployments are reliably disappointing. The gap is measurable, and the measurement is stark.

Published benchmark results show **the same model scoring 91.2% on Spider 1.0 and 21.3% on Spider 2.0** — a 70-point collapse on what is nominally the same task. On **BEAVER**, a benchmark built from real enterprise warehouse data, off-the-shelf models approach roughly **0%**.

The model did not get worse between those benchmarks. The databases got realistic.

	SPIDER 1.0	SPIDER 2.0
Tables	a handful	many
Columns per database	tens	~800 average
Column names	tidy	revenue , revenue_net , rev_final
Join paths	obvious	undocumented, tribal knowledge
Permissions	superuser	ignored by the benchmark

The documented causes are all about **context**, not capability:

- Ambiguous column names
- Undocumented join paths that exist only in someone's head
- Access controls benchmarks skip by running as superuser
- Dialect differences between database engines

The consensus prescription in the literature is **"fix the context, not the model."** This project takes that seriously and then tries to measure whether it is true.

4. The failure that actually matters

Almost every text-to-SQL evaluation reports one number: **execution accuracy** — the share of questions where the generated query returned the right rows.

That metric treats these two outcomes as equally bad:

the query errored	the user sees an error and asks again
the query returned 8,412	the user puts 8,412 in a report

They are not equally bad.

An error is **visible**. It announces itself, costs a retry, and nobody acts on it. A clean wrong number is **invisible** — and it propagates, into a slide, a board pack, a press release. Nobody re-checks a figure that arrived without an error attached.

This project's headline metric is the confidently-wrong rate: the share of questions answered with a clean, plausible, *wrong* figure. It is rarely measured, which is exactly why it is worth measuring.

Part II — The domain

The database models the **Indian Union Budget**. You do not need to know Indian public finance to follow this report, but six conventions matter, because the entire system is built around them.

5. Budget Estimate, Revised Estimate, Actuals

Every fiscal year has three different numbers for the same thing.

FIGURE	WHAT IT IS	WHEN IT EXISTS
Budget Estimate (BE)	What was proposed when the budget was presented, in February	Every year
Revised Estimate (RE)	The mid-year correction, published with the <i>next</i> year's budget	All but the newest year
Actuals	What was really spent, once the accounts close	Roughly two years in arrears

They differ routinely by 10–20%. So "*the budget for the Ministry of Railways in 2020-21*" is not one number. It is three, and the question does not say which.

6. Crore and lakh

Indian numbering uses **lakh** (100,000) and **crore** (10,000,000). One crore = 100 lakh. Budget documents mix them freely.

If a query sums a column measured in lakh and reports it as crore, the answer is **100× too large** — and still looks like a plausible budget figure.

7. Plan and Non-Plan, abolished in FY2017-18

Until 2016-17, Indian government spending was classified as "Plan" or "Non-Plan". From 2017-18 that classification was **abolished** and replaced by Scheme/Establishment.

A question asking for Plan expenditure in 2019-20 therefore has no answer. Not zero — *undefined*. The distinction matters enormously, because a query that returns zero looks like an answer.

8. Ministries are renamed and merged

- Ministry of Human Resource Development → Department of School Education and Literacy (2020)
- Ministry of Water Resources → merged into Jal Shakti (2019)
- Urban Development + Housing & Urban Poverty Alleviation → Housing and Urban Affairs (2017)

The **money** is continuous. The **name** is not. Filtering on one name across a decade produces a spending cliff that never happened.

9. Not every budget line is a ministry

The published budget lists **Interest Payments**, **Defence Pensions** and **Transfers to States** alongside actual ministries. These are real spending — about **44% of the total** — but they are not administrative bodies.

"Total expenditure" includes them. "What ministries spent" does not. On this data the two differ by **5.2×**.

10. The budget is organised by department, not ministry

There is no "Ministry of Education" line in the published budget. Its money appears under *Department of School Education and Literacy* and *Department of Higher Education*.

This was discovered while building the loader, not assumed — and it is why a question about "the education ministry" needs a certified reading rather than a guess.

Part III — The research that informed the design

Four findings from the literature shaped this project. Each is load-bearing: remove it and a design decision downstream stops making sense.

11. The benchmark-to-warehouse collapse

The 91.2% → 21.3% Spider 1.0/2.0 gap, and BEAVER's ~0%, establish that **the problem is not model capability**. This is the premise of the whole project. It is why the intervention is a semantic layer rather than a better model or a fine-tune.

12. "Fix the context, not the model"

The consensus remedy in the text-to-SQL literature is to repair the *context* the model is given: document what columns mean, certify metric definitions, write down valid join paths.

This directly produced the `semantic/` directory (Part IV) and the decision to make the ablation switch that directory on and off rather than swap models.

13. The semantic layer as an established pattern

A "semantic layer" is not a novel invention here — it is a standard component in business-intelligence tooling (dbt metrics, LookML, Cube), where its purpose is to stop two analysts computing "revenue" two different ways.

The contribution of this project is not inventing it, but **measuring what it is worth** when placed in front of a language model, on a schema deliberately built to punish its absence.

14. Selective prediction and abstention

In machine learning, **selective prediction** is the idea that a model may decline to predict when confidence is low, trading coverage for accuracy. Abstention is well studied in classification.

The equivalent for text-to-SQL is under-explored, and it motivated the central design choice here: an under-specified question should produce a *question back*, not a number. Part V.3 explains why that decision is made by rules rather than by the model.

Part IV — Architecture, module by module

The system is roughly 6,000 lines of Python across `src/rosetta/` (the library) and `scripts/` (entry points), plus three YAML files that hold the domain knowledge.

15. Data flow

```
question
|
v
ambiguity.assess()      rule-based, deterministic, BEFORE any model call
|
+-- AMBIGUOUS -----> ask which reading was meant
+-- UNANSWERABLE ---> explain why; do NOT return zero
|
v ANSWERABLE
generate.generate_sql() prompt = schema + semantic layer + question
|
v
correct.resolve()      EXPLAIN -> execute -> repair, max 3 attempts
|
v
db.Database            read-only, parse-verified, wall-clock timeout
|
v
answer + the SQL, always shown
```

15b. One question, end to end

The diagram above is abstract. Here is a real question travelling through the system, with the actual output both ways. This is the output of `scripts/demo.py`, reproduced in `docs/demo-output.txt`, not an illustration written for this report.

The question is deliberately ordinary — the kind anyone would type:

How much did the government spend in 2019-20?

Step 1 — Refusal rules run first, before any model call

`ambiguity.assess()` takes the raw text and checks it against hand-written rules. It never sees the database and never calls the model, so it is instant and gives the same verdict every time.

One rule fires, named `estimate_basis`: the question does not say **which of the three figures** it means. Budget Estimate, Revised Estimate and Actuals are three different numbers for 2019-20, routinely 10–20% apart (\$5).

The verdict is `AMBIGUOUS`, and the pipeline stops there. No prompt is built and no SQL is generated — the refusal costs nothing because it happens before the expensive part.

Step 2 — What each arm does with that verdict

Without the semantic layer, there are no rules to fire. The question goes straight to the model, which writes:

```
SELECT SUM(expenditure_crore) AS total_expenditure
FROM expenditure_actuals
WHERE fiscal_year = '2019-20'
```

That query is valid SQL. It runs without error. It returns:

666,197

No caveat, no question asked. The user has no way of knowing which of the three bases this is, or whether it includes Interest Payments.

With the semantic layer, the same question produces no SQL at all:

That question has more than one defensible answer here.

Which figure do you mean?

- Budget Estimate - what was proposed when the budget was presented
- Revised Estimate - the mid-year correction
- Actuals - what was really spent (available to 2021-22 only)

(these are three different numbers, routinely 10-20% apart)

Step 3 — Why the refusal was the right answer

The four certified readings of that one question return:

READING	FIGURE
Budget Estimate, all spending	2,905,387
Budget Estimate, ministries and departments only	1,636,754
Revised Estimate, all spending	2,872,216
Actuals, all spending	2,643,743

Four defensible answers, spanning a **1.8× range**. Any system that picks one silently is right at most a quarter of the time, and gives no signal when it is wrong.

Notice that these four differ along *two* axes, not one: which estimate basis, and whether "the government" means all spending or only ministries and departments (\$9, a 5.2× difference on this data). The rules caught the first and asked about it. The second is visible in the certified readings but was not itself raised as a question — an honest limit of keyword-based rules, and the kind of gap Part IX is careful not to paper over.

Now look back at what the baseline returned: **666,197**. That is not any of the four. Its closest certified reading is 1,636,754, which it misses by **59%**.

It did not pick the wrong reading from the menu. It produced a **fifth number**, and presented it exactly as it would have presented a correct one.

This is the failure the project is named after and built to measure. Nothing crashed. Nothing was flagged. A plausible figure with the right number of digits landed in front of the user, and the only way to discover it is wrong is to already know the answer.

Step 4 — What happens when the question *is* answerable

Pin down the ambiguity and the picture changes completely:

What was the Budget Estimate for the Ministry of Railways in 2019-20?

Now the rules pass it — one basis, one named entity, one year — and the rest of the pipeline runs:

- 1. `generate.generate_sql()` builds a prompt from the schema text, the semantic layer, and the question, and asks the model for SQL.
- 2. `correct.resolve()` runs `EXPLAIN` on the result first. `EXPLAIN` asks the database to *plan* the query without running it, which catches misspelled columns and bad joins cheaply. On a 182-column schema, most first attempts fail here on a column name.
- 3. If `EXPLAIN` fails, the error text is fed back to the model and it tries again — **three attempts maximum**, then the pipeline gives up and reports an error rather than guessing.
- 4. The surviving query executes inside `db.Database`, which is read-only, parse-verified as SELECT-only, and killed if it exceeds a wall-clock ceiling.
- 5. The answer is returned **together with the SQL that produced it**, always, so the reader can check the reasoning rather than trusting the number.

Step 5 is not decoration. A number you cannot audit is exactly the thing this project argues against.

16. The library

MODULE	RESPONSIBILITY
<code>db.py</code>	The execution sandbox. Nothing else touches the database. Guarantees read-only, SELECT-only (by parsing, not keyword matching), a wall-clock ceiling per query, and blocks 25 filesystem functions. Also does schema introspection.
<code>semantic.py</code>	Loads the three YAML files, renders the parts that bear on ambiguity into prompt text, reads exact column value-domains live from the database, and resolves entity lineage across renames by graph traversal.

MODULE	RESPONSIBILITY
<code>ambiguity.py</code>	The refusal rules. Classifies a question as ANSWERABLE / AMBIGUOUS / UNANSWERABLE and, under the DISCLOSE policy, converts defaultable ambiguities into stated assumptions. Pure functions, no model, no database.
<code>generate.py</code>	Prompt assembly and model calls. Defines a <code>Provider</code> protocol so a stub can replace the model in tests. Extracts a single SQL statement from whatever prose the model wrapped it in.
<code>correct.py</code>	The repair loop. Generate → EXPLAIN → execute → feed the error back, capped at three attempts. Records whether repair rescued each query.
<code>pipeline.py</code>	Orchestration. Holds the two switches that define the experiment: <code>use_semantic_layer</code> and <code>policy</code> .
<code>compare.py</code>	Result-set comparison. Canonicalises every cell, compares rows as a multiset, decides whether row order was actually requested.
<code>evaluate.py</code>	The outcome taxonomy and the metrics. Turns one pipeline answer into one of six labelled outcomes.

17. The domain knowledge

FILE	HOLDS
<code>semantic/glossary.yaml</code>	What each ambiguous column means, its unit, and the specific mistake it invites
<code>semantic/metrics.yaml</code>	Certified metric definitions, and which dimensions a question must pin before a metric can be computed
<code>semantic/joins.yaml</code>	Join paths classified certified / ambiguous / forbidden
<code>eval/questions.yaml</code>	The 60-question evaluation set with gold SQL

18. The entry points

SCRIPT	PURPOSE
<code>scripts/load_data.py</code>	Builds the database, deterministically, from one seed
<code>scripts/run_eval.py</code>	Runs the ablation arms and prints the comparison
<code>scripts/verify_gold.py</code>	Checks gold SQL, refusal-rule agreement, and all six data traps
<code>scripts/audit_gold.py</code>	Independently re-verifies all 32 gold queries by a different route
<code>scripts/adjudicate.py</code>	Evidence behind the 28 refusal categorisations
<code>scripts/measure_variance.py</code>	How much of the result is harness noise
<code>scripts/demo.py</code>	The side-by-side contrast on one ambiguous question

SCRIPT	PURPOSE
<code>scripts/ask.py</code>	Interactive querying

Part V — Design decisions and why

19. Why a synthetic-detail database rather than real data

Decision. Entity names, entity count and every spending magnitude come from the published Union Budget. The row-level detail beneath them — scheme, account-head, state and month rows — is generated, normalised so it sums back to the real figures.

Reasoning. The real published file is entity-level only; it has no scheme or monthly breakdown. Building the traps required detail that does not exist publicly. The trade buys a repository that rebuilds from one command with no external download, at the cost of not being able to claim the figures are real.

Honesty requirement. This must be stated wherever the project is described. "Real structure and magnitudes, synthetic detail" is defensible; letting a reader assume otherwise is not.

20. Why deliberately ambiguous columns

A schema that is merely *large* does not test anything interesting. Six ambiguities were designed in, each modelled on a real property of the published data, and each with the same shape: **a query that runs cleanly and returns a plausible wrong number.**

TRAP	THE MISTAKE	COST
Units	Summing <code>be_amount_lakh</code> instead of <code>be_amount_crore</code>	100× too high
Estimate basis	Answering with BE when the question meant Actuals	10–20% out
Plan era	Summing <code>plan_amount_crore</code> across all ten years, NULL for seven	7 years silently dropped
Entity scope	Counting functional lines as ministries, or excluding them from a total	5.2× apart
Renames	Filtering one ministry name across a rename boundary	series truncated
Grain / fan-out	Joining annual to monthly without aggregating first	33× inflated

All six are re-verified on every build by `verify_gold.py`, so those figures are measured rather than asserted.

21. Why refusal is rule-based, not model-based

This looks backwards at first — the system uses a language model for the hard part (writing SQL) and hand-written rules for the subtle part (deciding whether a question is answerable). Three reasons:

1. **Determinism.** If refusal came from a model, the headline number would move between runs for reasons unrelated to what is being measured. Measured effect: the semantic arm has **0.0 points of spread** across three runs; the baseline has 5.0.
2. **Auditability.** Every refusal traces to a named rule, so it can be defended to the person who got refused.
3. **Cost.** It runs before generation, so it is free when it fires. 28 of 60 questions never reach the model.

The underlying principle: **the decision not to answer is not delegated to the thing being guarded against.**

22. Why EXPLAIN before execute

EXPLAIN asks the database to *plan* a query without running it. It catches unknown columns, bad joins and type errors for the price of a plan, before the query spends its execution budget.

On a 182-column schema most first attempts fail on a column name, so catching that cheaply is most of the loop's value.

23. Why multiset comparison

Two result sets can be "the same answer" while differing in a dozen boring ways — column order, row order, integers as floats, NULL vs NaN, whitespace. Every cell is canonicalised first.

Rows are then compared as a **multiset** — a bag that counts duplicates — not a set. A set would let a query that lost its **GROUP BY** compare equal to one that did not.

24. Why six outcomes instead of one accuracy number

Collapsing everything into execution accuracy hides the distinction the project exists to measure. The taxonomy is:

OUTCOME	MEANING
Correct	Answered, result matches gold
Confidently wrong	Clean, plausible, wrong number ← the headline
Answered + disclosed	Answered an under-specified question using a stated default, and said so
Wrong order	Right values, wrong sequence
Errored	Failed after repairs — visible, therefore safe
Refused rightly / wrongly	Declined correctly / declined something answerable

Two deliberate choices inside it:

- **Wrong order is not confidently wrong.** A reader who gets correct rows in a different sequence has not been misled about any figure. Folding it in would inflate the result in the project's own favour.
- **Refused-wrongly is tracked on purpose.** A system that refuses everything scores zero confidently-wrong while being useless. The pair must be read together.

25. Why local inference

The eval loop makes several hundred model calls per pass and is run many times. A rate-limited API tier would throttle every run. Local inference via Ollama removes rate limits, needs no key, and costs nothing — at the price of

lower absolute SQL quality, which is acceptable because the project measures a *delta*, not an absolute.

26. Why the ablation holds the schema identical

The two arms differ by exactly one argument. Schema text, instructions, question and repair context are byte-identical between them. Without that discipline, any measured difference could be attributed to prompt length rather than to the semantic layer.

Part VI — Decisions that turned out wrong

This is the most useful section in the report. Everything here was believed correct when written, and was changed after evidence contradicted it.

Three failure *shapes* recur. They are worth naming, because they generalise well beyond this project.

Shape A — Agreement is not correctness. Two artefacts written by the same author from the same assumption will agree with each other while both being wrong.

Shape B — A rule that is right for the wrong reason looks identical to one that is right. Until something removes the cover.

Shape C — A number that moves is not a number. Any improvement inside the measurement noise is not an improvement.

27. Excluding functional lines from the entity list — *wrong*

Decision. Treat only ministries and departments as spending entities; exclude Interest Payments, Defence Pensions, Transfers to States.

Why it was wrong. Those lines are ~44% of the budget. Excluding them made the database total ₹23.7 lakh crore against a real ₹47.7 lakh crore — the provenance claim that magnitudes are real stopped holding.

Replaced by. Include them, tagged with `entity_kind`. This turned a bug into a feature: "total expenditure" vs "what ministries spent" became a genuine certified-reading problem worth 5.2×.

28. Leaving `department_id` unpopulated — *wrong*

Decision. Leave the column NULL; nothing needed it.

Why it was wrong. A 100%-NULL foreign key means any join through it returns nothing, and `SUM` over nothing returns `NULL` — which reads as a clean answer. The model joined through it and produced exactly that. The failure was real but caused by a dangling column rather than by any designed ambiguity, so it would have dominated the headline for the wrong reason.

Replaced by. Populated it. Both join paths now agree.

29. A ministry keyword that matched nothing — *wrong*

Decision. Attach schemes to ministries by keyword; use `"jal shakti"` for water schemes.

Why it was wrong. There is no "Ministry of Jal Shakti" row in the published budget — the money sits under *Department of Drinking Water and Sanitation*. Two schemes silently ended up with zero allocations, and a gold query returned NULL.

Replaced by. Repointed to the real department name. **Shape B:** the code was not broken, it was matching against an assumption about the data rather than the data.

30. Generating Revised Estimates for every year — *wrong, and it hid for a long time*

Decision. `fiscal_years.re_available` marks the newest year as having no RE. Separately, the loader generated `re_amount_crore` for every year.

Why it was wrong. The flag and the data disagreed. Two evaluation questions asserted "no Revised Estimate exists for 2023-24" while the column held **1,167 values**.

How it survived. `verify_gold.py` reported **60/60 agreement** between the refusal rules and the gold set — and both were wrong in exactly the same way, because both were written from the same assumption. **This is Shape A in its purest form.** Checking each against *the data* found it in seconds.

Replaced by. Both the flag and every RE column now derive from one `re_published()` function, so they cannot drift apart.

31. Counting "wrong order" as confidently wrong — *wrong, and it flattered my own number*

Decision. Any result not matching gold counted as confidently wrong.

Why it was wrong. Right rows in a different sequence has not misled anyone about a figure. Counting it inflated the headline in the project's favour — the one direction an evaluation cannot afford to err.

Replaced by. A separate `WRONG_ORDER` outcome, excluded from the headline and from `CORRECT`.

32. Hardcoding a spread in the demo — *wrong*

Decision. The demo script printed "spanning a 5x range".

Why it was wrong. The actual range was 1.8x. A demo for a project about unverified numbers was asserting an unverified number.

Replaced by. The demo computes the spread from live query results, and additionally checks whether the baseline's answer matches *any* certified reading — it did not; it had produced a fifth number.

33. A semantic layer that only surfaced part of itself — *wrong*

Decision. Render the semantic layer into the prompt.

Why it was wrong. `semantic_context()` surfaced only five of the eight documented ambiguities. Gross/net, borrowings and devolution were documented in the glossary and never reached the model. Three trap failures traced to exactly that gap.

Replaced by. Expanded coverage, plus column value-domains read **live from the database** so they cannot go stale, plus an explicit NULL-semantics section.

Follow-up finding. Expanding it did **not** move the headline. It fixed two specific behaviours and the net was zero, inside the noise. Reported as a behavioural improvement with no measured effect — **Shape C**.

34. Framing refusal as a binary — *wrong*

Decision. A question is either answered or refused.

Why it was wrong. The failure being measured is a *confident* wrong number. A figure delivered with its assumption stated is not confident, so it is not that failure. The binary framing missed a third option.

Replaced by. A switchable policy: STRICT refuses; DISCLOSE answers with a certified default and names the assumption. Both measured. A dimension only gets a default where convention supplies one — `estimate_basis` does, `scheme_identity` does not, because Swachh Bharat Gramin and Urban are two different schemes and picking one silently is the error, not the fix.

Result: refusal rate 46.7% → 30.0%, confidently wrong 16.7% → 21.7%. A real trade, honestly reported, with STRICT kept as the default.

35. Trusting a read-only connection as a sandbox — *wrong, and security-relevant*

Decision. A read-only DuckDB connection plus SELECT-only parsing is sufficient isolation.

Why it was wrong. A read-only connection blocks *writes* and nothing else. This is a pure SELECT that passes every structural check:

```
SELECT * FROM read_csv_auto('C:/Users/me/secrets.csv')
```

Confirmed by planting a value in a file outside the database and reading it back. **A language model writes this SQL** — a model that is confused, or steered by text in the schema it was shown, can emit one of these with nothing upstream noticing.

Replaced by. A block on 25 filesystem and external-catalog functions.

The fix was then wrong twice, and only testing caught it:

1. sqlglot models some of these as *typed* nodes (`read_parquet` → `exp.ReadParquet`), so an `Anonymous` -only check missed them.
2. On a typed node, `.name` returns the **first argument** — so `read_csv('x.csv')` reported itself as `"x.csv"` and slipped through again.

36. My own audit script contained the bug it was auditing — *wrong*

While writing independent cross-checks for the gold queries, five flagged. All five were bugs in the **audit**, not in gold. The most instructive:

- A cross-check rewrote `effective_to_year >= 2016` as `NOT (effective_to_year < 2016)`. In SQL, `NOT (NULL < 2016)` evaluates to `NULL`, not `TRUE` — silently dropping 102 of 107 rows. **The audit for a project about SQL traps fell into a SQL trap.**
- A check asserted no special-category state exceeds 20M population. Assam is 31.2M. The prior was simply wrong about the world.
- A tolerance of `1e-9` was tighter than accumulated floating-point rounding across ~1,000 rows can deliver.

37. An adjudication script decoupled from its own input — *wrong*

Decision. `adjudicate.py` loads `eval/questions.yaml`, then checks hardcoded lists.

Why it was wrong. It loaded the file and **never used it**. Renaming or adding a question would leave it reporting "12/12 verified" against a stale list forever. **Shape A again**, in a script written specifically to break Shape A.

Replaced by. It reconciles its lists against the YAML and refuses to run on drift. Verified by simulating a rename.

38. Prose that drifted from the data — *wrong, twice*

The README quoted a fan-out factor of 42×, then 35×, while the live value was 33×. The seed is fixed, but any change to *how many random draws occur* shifts the whole downstream stream, so generated figures move between revisions.

Separately, `variance.json` was overwritten by a later run and the README kept quoting the earlier figures — baseline 76.7% with 0.0 spread when the file said 76.1% with 5.0. The effect/noise ratio was being reported as 35.7× when it was 11.9×.

Replaced by. Headline figures injected from the results files by script, plus a caveat that generated figures move between revisions and `verify_gold.py`'s live output is authoritative over prose.

Part VII — Measurement and results

39. The evaluation set

60 questions in four categories:

CATEGORY	COUNT	CORRECT BEHAVIOUR
Answerable	20	answer, matching a hand-written gold query
Trap	12	answer, resisting a tempting wrong path that runs cleanly
Ambiguous	16	refuse — more than one certified reading
Unanswerable	12	refuse — no reading has data

32 questions carry gold SQL. All 32 were independently re-verified by a *different* route — an alternate join path, an algebraic identity, or a known generation invariant — rather than by re-reading them.

40. The headline result

Same model, same database, same schema text. The only difference is whether `semantic/` is loaded.

METRIC	BASELINE	WITH LAYER	CHANGE
Confidently wrong	76.1%	16.7%	−59.4 pts
Execution accuracy	32.3%	53.1%	+20.8 pts

METRIC	BASELINE	WITH LAYER	CHANGE
Refusal recall	0%	100%	+100 pts

Both are means of three runs.

41. Is that number real?

A headline quoted without its spread is not a falsifiable claim. Each arm was run three times at temperature 0.

	BASELINE	WITH LAYER
Confidently wrong	76.1% mean, 73.3–78.3%	16.7% every run
Spread	5.0 pts	0.0 pts
Questions with an unstable outcome	3 of 60	0 of 60

Effect 59.4 points against noise 5.0 points — a ratio of 11.9×

The asymmetry is itself a finding: deterministic pre-generation refusal makes the semantic arm perfectly stable, while the baseline swings across five points because local inference is not bit-reproducible across server restarts.

The rule this establishes: any claimed improvement smaller than about 5 points is not distinguishable from noise. That rule has already caught one false positive in this project (§33).

42. Two models, one harness

	QWEN2.5-CODER:7B	LLAMA3.1:8B
Confidently wrong, baseline	76.7%	76.7%
Confidently wrong, with layer	16.7%	35.0%
Execution accuracy, with layer	53.1%	18.8%
Refusal recall	100%	100%

The most interesting result in the project. The two baselines are *identical*. Swapping a general model for a code model is worth 34 points of execution accuracy once the layer is on, and moves the baseline confidently-wrong rate by **exactly nothing**.

The confidently-wrong rate is a property of **the schema's ambiguity**, not of the model's competence. A better model does not rescue you from it.

But the layer does not close the whole gap — after it, the two models are still 18 points apart. So the honest version of "fix the context, not the model" is narrower than the slogan: **context fixes the ambiguity half; model capability still owns the correctness half**.

43. Policy comparison

	BASELINE	STRICT	DISCLOSE
Confidently wrong	75.0%	16.7%	21.7%
Refusal rate	0%	46.7%	30.0%

	BASELINE	STRICT	DISCLOSE
Answered with a stated assumption	0	0	8

DISCLOSE answers 10 questions STRICT refuses; 8 match the certified default exactly. The refusal rate falls 16.7 points — a system that refuses half of what it is asked stops being asked.

The cost is a new failure mode STRICT does not have: **the stated assumption and the computed figure can disagree**. A mislabel is worse than a refusal, because the label makes it look checked. STRICT remains the default.

Part VIII — Running and extending

44. Running it

```
pip install duckdb sqlglot pandas pyyaml
```

```
python scripts/load_data.py      # build the database, deterministic from one seed
python scripts/verify_gold.py    # gold queries, refusal rules, all six traps
python scripts/run_eval.py       # both arms, prints the ablation
```

Generation needs a local Ollama server with a model pulled. `qwen2.5-coder:7b` fits in about 6 GB of VRAM.

```
python scripts/run_eval.py --arm all --model llama3.1:8b
python scripts/measure_variance.py --repeats 5
python scripts/demo.py
python scripts/ask.py "What was the Budget Estimate for the Ministry of Railways in 2019-20?"
```

45. Extending it

Add a question. Append to `eval/questions.yaml` with a category and, if answerable, gold SQL. Then run `verify_gold.py` (does it execute?), `audit_gold.py` (does an independent route agree?) and `adjudicate.py` (does it reconcile?). The last will refuse to run until its lists are updated — that is deliberate.

Add an ambiguity dimension. Add it to `semantic/metrics.yaml`, add a detection rule in `ambiguity.py`, and decide whether it belongs in `CERTIFIED_DEFAULTS`. It belongs there only if convention supplies an answer; if the alternatives are different *objects* rather than different framings of one object, it must not have a default.

Swap the model. `--model <tag>` for any Ollama model. For a non-Ollama provider, implement the `Provider` protocol in `generate.py` — one method, `complete(prompt, temperature)`.

Change the refusal policy. `--arm disclose`. The behaviour is defined by `CERTIFIED_DEFAULTS` in `ambiguity.py` and the pin text in `pipeline.py`; those two must agree, or the disclosure and the figure diverge.

Port to another database. `db.py` is the only module that touches DuckDB. The read-only guard uses `sqlglot`, which supports many dialects — change `dialect="duckdb"` and the `_FILESYSTEM_FUNCTIONS` list, which is engine-specific.

Part IX — Limitations

- **Row-level data is synthetic.** Entity names, entity count and every magnitude are real; the detail beneath them is generated. Do not describe this as "real government data" without that qualification.
- **100% refusal recall is against *this* question set.** A set written by someone else would score lower. The rules are keyword-based, which buys determinism and auditability at the cost of missing paraphrases a model would catch.
- **One SQL dialect.** DuckDB only. Multi-dialect compilation was cut for scope.
- **No row-level security.** Every query runs as the same read-only role. Real warehouses enforce per-user visibility, and benchmarks that ignore it overstate accuracy.
- **Reproducible per commit, not across commits.** A change to how many random draws occur shifts the whole downstream data stream. Trust `verify_gold.py`'s live output over any number in prose.
- **The 16 ambiguous categorisations rest on a measured threshold, not a proof.** Every one conceals a discrepancy of at least 1.14×; whether refusing is the right response at 1.14× is a product judgement.
- **DISCLOSE can mislabel.** The stated assumption and the computed figure can disagree (2 of 60 with a 7B generator). STRICT has no such failure mode.

Glossary

TERM	MEANING
Ablation	An experiment that removes one component and holds everything else fixed, so any change is attributable to that component
Abstention	A system declining to predict rather than guessing, trading coverage for accuracy
API	A way for one program to ask another for something over a network. Using a model by API means sending your text to someone else's server
Arm	One configuration in a comparison. This project has two: with the semantic layer and without
Baseline	The arm you are measuring against — here, the system with no semantic layer
BE / RE / Actuals	Budget Estimate, Revised Estimate, Actuals — three different figures for the same year
Benchmark	A fixed set of questions with known-correct answers, used to compare systems fairly. Spider and BEAVER are the well-known ones for text-to-SQL
Column	One attribute that every row in a table has, such as <code>fiscal_year</code>
Confidently wrong	A query that runs cleanly and returns a plausible but incorrect number. The failure this project measures
Crore / lakh	Indian units: 1 crore = 10,000,000; 1 lakh = 100,000; 1 crore = 100 lakh

TERM	MEANING
CTE	Common Table Expression — a named intermediate result inside a query, written <code>WITH x AS (...)</code>
Deterministic	Same input, same output, every time. Hand-written rules are deterministic; language models are not
EXPLAIN	A SQL command that plans a query without running it; used here as a cheap validity gate
Execution accuracy	Share of questions where the generated query returned the right rows. The conventional single metric
Fan-out	A join that multiplies rows because the tables are at different grains, inflating any sum
Gold query	A hand-written correct SQL query for an evaluation question, used as the reference answer
Grain	What one row of a table represents — here, one year versus one month
Hallucination	A model stating something fabricated in the same confident tone it uses for correct answers
Join	Combining two tables by matching a shared value. Joining tables of different grain causes fan-out
Large language model (LLM)	A program that predicts likely text. It generates plausible continuations; it does not look facts up
Local inference	Running a model on your own hardware instead of calling a hosted API. No key, no rate limit, no per-call cost
Multiset	A collection that counts duplicates; used for row comparison so a lost <code>GROUP BY</code> cannot pass
NULL	SQL's marker for "no value recorded". Not zero, and it poisons comparisons — see three-valued logic
Ollama	The tool used here to run a language model locally
Parameters (as in 7B)	The internal numbers a model learned in training. 7B means 7 billion — small enough to run on an ordinary machine
Probabilistic	Producing output by sampling from likelihoods, so repeated runs can differ. The opposite of deterministic
Prompt	The text handed to the model. Here: schema description + semantic layer (when on) + the user's question
Quantisation	Compressing a model's weights to fewer bits so it fits in less memory, at some cost to quality
Row	One fact in a table — one ministry's allocation in one year
Schema	The catalogue of a database: which tables exist, which columns each has, and of what type
Seed	The fixed starting number that makes generated data reproducible; the same seed rebuilds byte-identical data
Selective prediction	Allowing a model to decline low-confidence cases; the ML framing of this project's refusal behaviour

TERM	MEANING
Semantic layer	A written, versioned description of what data means: column glossary, certified metrics, validated join paths
Spread	The gap between the highest and lowest result across repeated runs — the noise floor a real effect must clear
SQL	The language used to ask a database for data
Table	A grid of rows and columns holding one kind of fact
Temperature	How adventurous the model's word choice is. 0 means always take the most likely option; every measurement here uses 0
Text-to-SQL	Turning a natural-language question into a database query automatically
Three-valued logic	SQL's TRUE / FALSE / NULL. <code>NULL < 2016</code> is NULL, not FALSE — the cause of a real bug in §36
YAML	A plain-text format for structured data, indentation-nested, meant to be written and reviewed by people

Compiled from the repository at commit `efd015b + uncommitted changes`. All measurements produced by `scripts/run_eval.py` and `scripts/measure_variance.py`, stored in `results/`. Where a figure is a mean, its range is given alongside. Rendered to PDF by `scripts/build_report.py`.